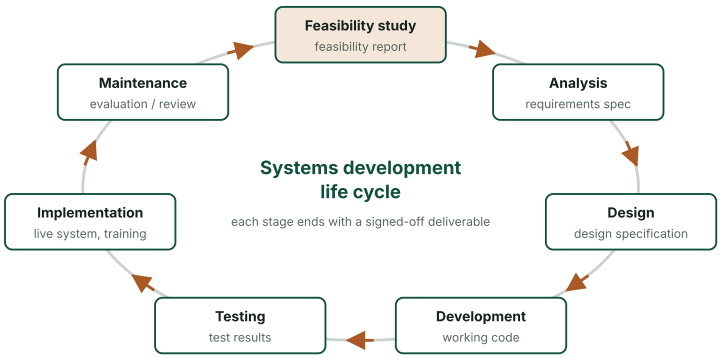
# Why develop new systems? Systems, the software crisis and the life cycle

AP01 FACT FILE

**Specification: students should be able to**

- Describe the characteristics of a system and explain how large systems are made up of sub-systems
- Understand that a computer system consists of a user interface, processes and data, and explain the factors that affect the choice of user interface
- Explain the reasons why an organisation may need to develop a new information system
- Explain the impact of the software crisis and the emergence of software engineering
- Describe the stages of the systems development life cycle (SDLC), the purpose of each stage and why a structured approach is used

**Key words:** system, sub-system, computer system, user interface (UI), process, data, information, legacy system, software crisis, software engineering, user requirements, systems development life cycle (SDLC), feasibility, analysis, design, maintenance, evaluation, deliverable




The systems development life cycle: seven stages arranged in a cycle, each labelled with its main deliverable

## Systems and why we develop them

A system is a collection of interdependent components that work together to achieve a purpose. Every system has three characteristics: a defined purpose, a defined need that it meets, and a number of components that rely on one another. Large systems, such as the computer system of a health trust or an airline, are usually too complex to understand as a whole, so they are broken down into smaller sub-systems, such as appointments, pharmacy and payroll, which pass data between them. In this course, systems development means creating systems that involve computer processing. Organisations develop new systems when the current one no longer serves its purpose or meets its need: the business has grown, laws have changed, competitors offer better services, technology has moved on, or the old system is unreliable. Because software development went badly wrong in its early years, a period known as the software crisis, the industry adopted a structured, engineering approach, which today is represented by the systems development life cycle. This lesson covers all of these ideas, which form the foundation for the rest of AS 1.

## Data, user interface and processes

A computer system uses computing power to support a function. It must be able to receive data, process it, store both the data received and the results of processing, turn processed data into information that people can use, and pass data to and from linked sub-systems. Data are raw facts and figures without context; information is data that has been processed and given meaning, such as a low-balance alert or a monthly sales report. Any computer system can therefore be described as three elements. The data is what the system holds and works on, whether entered by users or generated by processes. The user interface is how users send data into the system and see the results. The processes are the operations that manipulate the data, such as calculating, validating, searching, sorting and updating records. Each element matters in its own right: inaccurate or incomplete data produces unreliable output; a poor interface leads users to make errors, take longer or avoid the system altogether; and poor processing produces incorrect results or wastes time. When answering questions, always give concrete examples of each element for the scenario.

### Choosing a suitable user interface

The user interface is the gateway between a system and its users, so its usability often decides whether a system succeeds. The type of interface chosen depends on three things. The first is the application: how much data must be input and what output is required. A self-service kiosk at a cinema needs only a few taps, whereas a nurse updating a patient record must complete many fields, so a keyboard, mouse and form-based screen suit that task. The second is the environment: a busy, noisy shop floor, a moving vehicle or an outdoor car park each impose limits. A satnav takes a destination by touchscreen while the car is stopped, but gives directions by voice while driving so that the driver keeps their eyes on the road. The third is the characteristics of the users: trained staff can cope with a complex interface, but members of the public need something simple and intuitive, with accessibility features for users with disabilities. Interfaces also evolve with technology and demand, from manual payment to contactless cards, barcode scanning, voice assistants and automatic number-plate recognition.

### Reasons for a new system

Organisations develop new systems when the current system no longer meets their needs. Internal reasons include growth in the volume of data or transactions, inefficient manual processes, duplicated data entry and the errors that follow, unreliable or unsupported legacy hardware and software, and the need for better management information to support decisions. External reasons include changes in legislation, such as data protection or tax reporting rules, pressure from competitors, rising customer expectations of online and mobile access, and the arrival of new, affordable technology. A good answer always links the reason to its effect on the organisation, for example: the paper diary cannot be used by two members of staff at once, so double bookings occur and customers are lost. Simply saying that a new system would be faster or better rarely earns credit unless you explain what the improvement means in that context. A new system has to satisfy the organisation that operates it as well as the individuals who use it, which is covered further in AP02.

### The software crisis and software engineering

In the early decades of computing, software was written informally, with few agreed methods. From the mid-1960s, demand for larger and more complex systems grew rapidly, and the industry could not keep up; the term software crisis became widely used after the 1968 NATO Software Engineering Conference held in Germany. Several factors caused the crisis. Developers had little experience of similar projects on which to base estimates, so time and cost estimates were crude and usually wrong. Development tools were primitive, so software took a long time to write, and there were no tools to help detect errors early, which made faults slow and expensive to find and systems costly to maintain. Methods for capturing requirements were imprecise, so systems often failed to do what users needed, and there were no specialist testers or formal testing process. The impact was software that was late, over budget, unreliable and sometimes abandoned. The response was software engineering: treating the design, construction and upkeep of software as a disciplined, systematic engineering activity, in the way civil or mechanical engineers work, so that quality software is delivered on time, within budget and to the user's requirements.

### The systems development life cycle

The SDLC is a structured framework that breaks the development of an information system into stages: feasibility study, analysis, design, development, testing, implementation, and maintenance and evaluation. Each stage has clear objectives and deliverables, such as the feasibility report, requirements specification, design specification, test results and user documentation, and these are usually signed off by the client before the next stage begins. Documentation is produced throughout, and the finished system is formally handed over to the client. Precise definition of the user requirements is critical: an error made at this stage may only be discovered after coding, when it is expensive to put right because design, code and tests must all be redone. Breaking the project into stages allows managers to estimate costs, plan and monitor progress against milestones. The main weakness of a strict sequence is inflexibility: if the client's needs change late in the project, returning to earlier stages is slow and costly, which is why approaches such as prototyping and agile methods exist. The process is called a cycle because evaluation and maintenance of one system eventually reveal the need for its replacement.

## T Key terms from the mark schemes

**System**
A set of interdependent components that work together for a stated purpose, satisfying an identified need.
`Credited: Something with a purpose, a need and a number of components`

**Sub-system**
A smaller system that forms part of a larger system and performs one part of its overall function, often exchanging data with other sub-systems.

**User interface (UI)**
The means by which a user interacts with a computer system, entering data into it and viewing the results of processing.
`Credited: How the user interacts with the system`

**Data**
Raw facts and figures that have no context or meaning on their own, which are input to or stored by a system.
`Credited: Raw facts`

**Process**
An operation carried out by a computer system that manipulates data, for example calculating, sorting, searching or updating, converting data into information.
`Credited: A task carried out by the system`

**Software crisis**
The period from the mid-1960s when software projects were frequently late, over budget, unreliable and failed to meet user requirements because development methods could not keep pace with demand.

**Software engineering**
Using a disciplined, systematic engineering process to design, build and maintain software.

**Systems development life cycle (SDLC)**
A structured framework of stages (feasibility, analysis, design, development, testing, implementation, maintenance and evaluation) used to plan, build and maintain an information system.

**Legacy system**
An old system that is still in use but is often unreliable, unsupported or expensive to maintain.

**Deliverable**
A document or product, such as a feasibility report or test plan, that is produced and signed off at the end of a stage of a project.

**Information system**
A system of people, procedures, hardware, software and data that collects, processes, stores and distributes information to support an organisation.

**Analysis**
The stage in which the current system is investigated using fact-finding methods to establish the problems and the requirements of the new system.

## Exam focus

State or list questions want short points, one mark each, such as naming a characteristic of a system or a function of a computer system. Describe questions need the point plus what it involves, for example: Analysis, the current system is investigated using fact-finding methods to establish the user requirements (2). Explain questions need a reason or consequence, so use because or so that. For the software crisis, mark schemes credit specific causes: little experience so inaccurate estimates, unsophisticated tools, no early error detection, systems not meeting user requirements, and no specialist testing; and they credit software engineering as a systematic or structured approach to design, development and maintenance. When asked for the data, user interface and processes of a scenario, give items specific to it, such as product ID and quantity in stock for a stock system, rather than generic words. Always apply answers to the context; do not confuse testing (checking the system works) with evaluation (judging whether it met its objectives once in use), or data with information.

## ? Check yourself

1. A Northern Ireland bus operator uses a real-time passenger information system at its stops. State the three characteristics that this, like every system, must have. [3]
2. A leisure centre run by a Northern Ireland council uses a computer system to manage memberships, class bookings and payments. Identify the purpose, the need and two components of this system. [4]
3. A newsagent in Omagh uses a computer system for stock control. For this system, describe one example of the data it holds, one example of its user interface and one example of a process it carries out. [6]

**Answers:** 1. A defined purpose, e.g. to tell passengers when the next bus will arrive (1) / meets a defined need, e.g. passengers waiting without knowing about delays (1) / a number of interdependent components, e.g. GPS units on buses, a central server, display screens at stops (1). Accept the characteristics without examples. 2. Purpose, e.g. to manage memberships and allow customers to book classes and pay (1) / Need, e.g. high volume of bookings cannot be handled manually; avoid overbooked classes (1) / Any two components, e.g. membership database (1), online booking website/app (1), card payment terminal, access-control turnstiles, staff PCs, network 3. Data (max 2): an appropriate data item, e.g. product barcode/ID, description, quantity in stock, reorder level, price (1) with description of where it comes from/why it is held (1) / User interface (max 2): e.g. barcode scanner at till / stock screen to view and edit levels / form to add new products (1) with how it is used (1) / Processes (max 2): e.g. reduce stock quantity when an item is sold, increase it when a delivery arrives, compare quantity with reorder level and produce reorder list, produce sales reports (1) with description (1)